

Recurrent Neural Network

Dept. of CSE, University of Asia Pacific

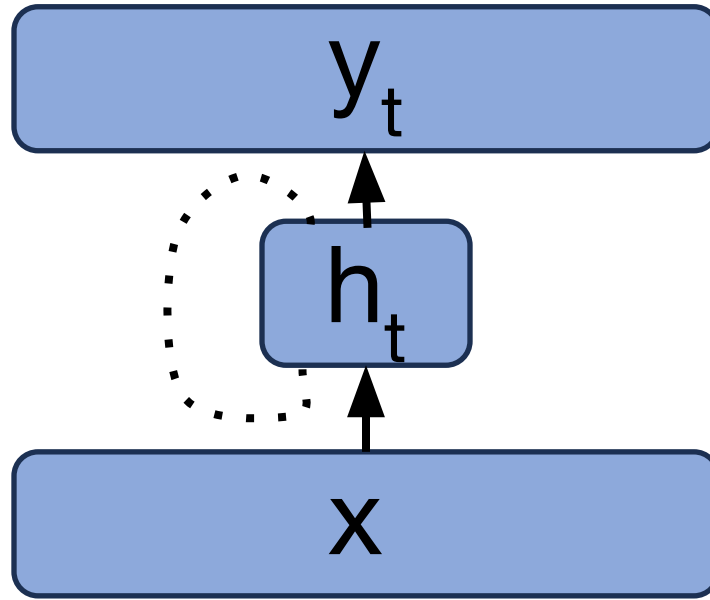
Modeling Time in Neural Networks

- Many real-world problems involve **sequences over time**
 - Examples: sentences, speech, stock prices, sensor data
- Standard neural networks struggle to remember previous inputs
- **Recurrent Neural Networks (RNNs)** add memory to deep learning models
- RNNs are used in applications like language modeling, speech recognition, and stock market forecasting

Recurrent Neural Networks (RNNs)

- Any network that contains a cycle within its network connections.
- The value of some unit is directly, or indirectly, dependent on its own earlier outputs as an input.

Simple Recurrent Nets

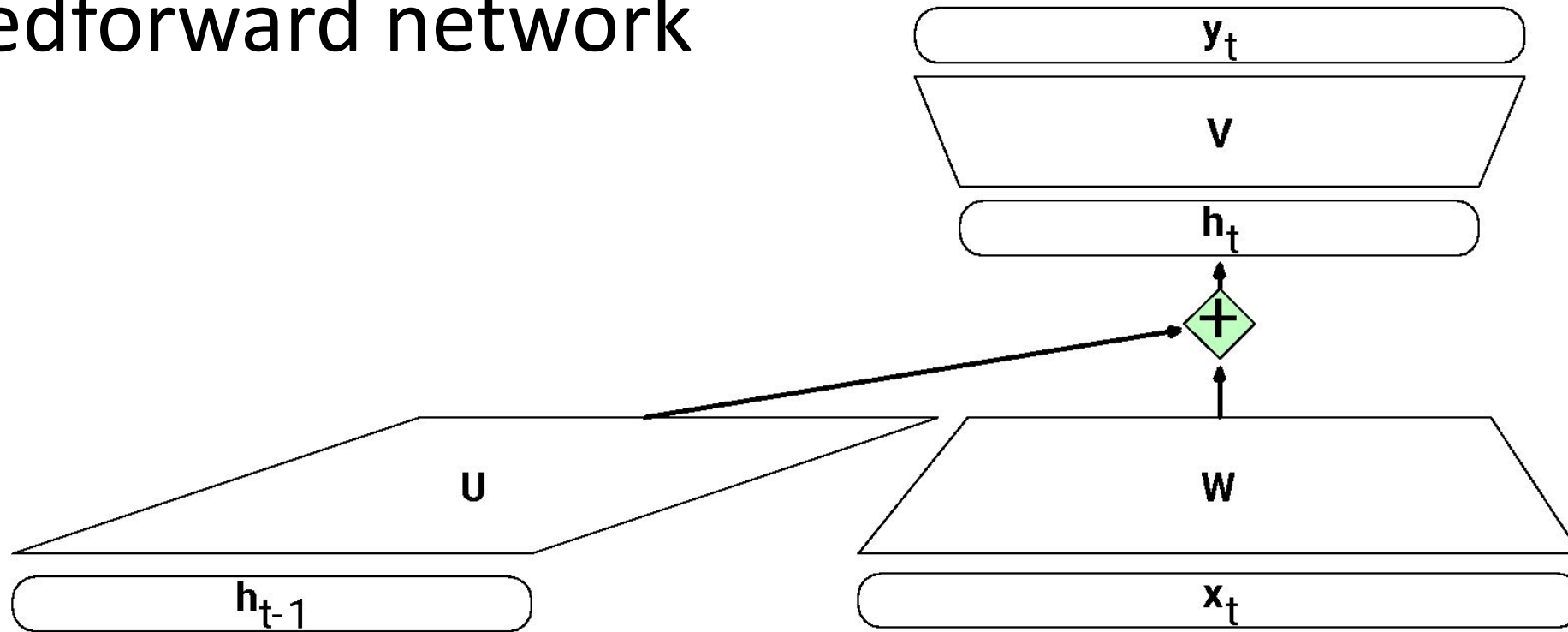


- The hidden layer has a t recurrence as part of its input
- The activation value h_t depends on x_t but also

Forward inference in simple RNNs

- Very similar to the feedforward networks we've seen!

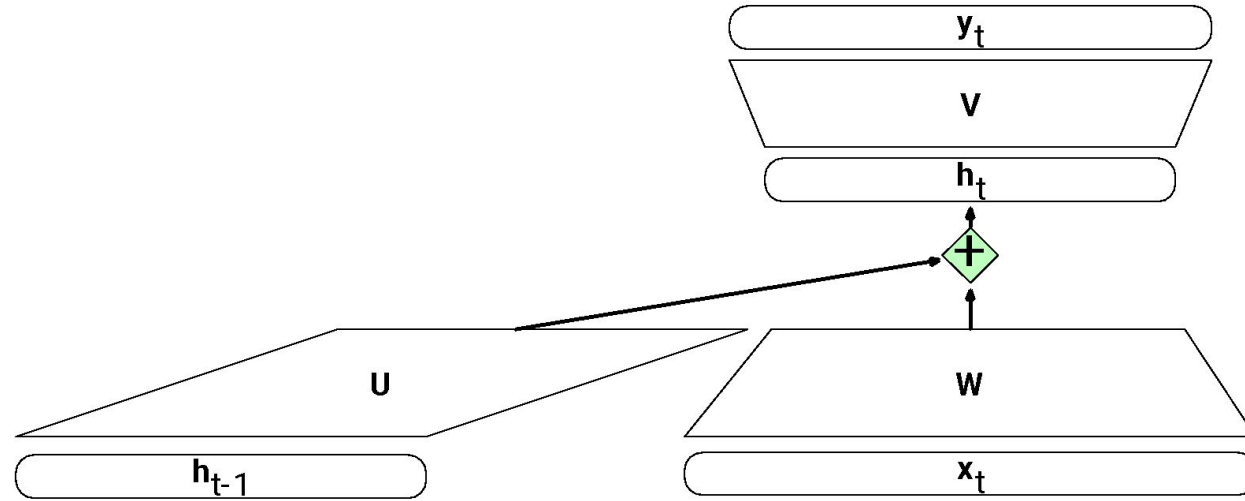
Simple recurrent neural network illustrated as a feedforward network



$$h_t = g(U h_{t-1} + W x_t)$$

$$y_t = \text{softmax}(V h_t)$$

Simple recurrent neural network illustrated as a feedforward network



- W , the weights from the input layer to the hidden layer,
- U , the weights from the previous hidden layer to the current hidden layer,
- V , the weights from the hidden layer to the output layer.

$$h_t = g(U h_{t-1} + W x_t)$$

$$y_t = \text{softmax}(V h_t)$$

Inference has to be incremental

- Computing h at time t requires that we first computed h at the previous time step!

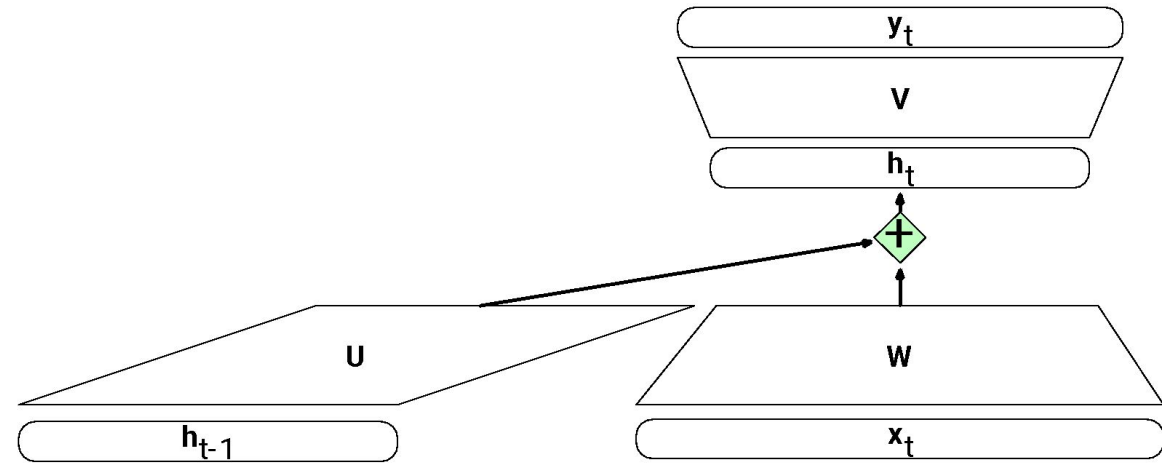
function FORWARDRNN(x , *network*) returns output sequence y

```
h0 ← 0
for  $i$  ← 1 to LENGTH( $x$ ) do
    h $i$  ←  $g(Uh_{i-1} + Wx_i)$ 
    y $i$  ←  $f(Vh_i)$ 
return  $y$ 
```


Training in simple RNNs

Just like feedforward training:

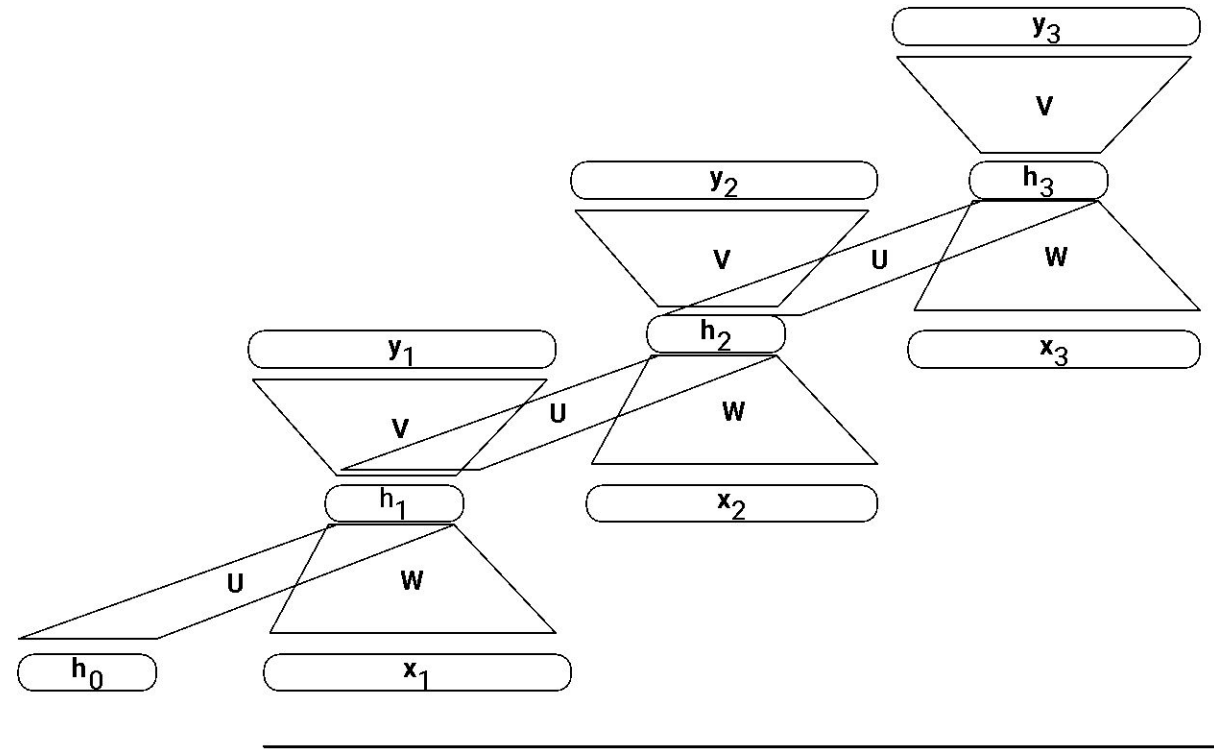
- training set,
 - a loss function,
 - backpropagation
-
- Weights that need to be updated:
 - **W**, the weights from the input layer to the hidden layer,
 - **U**, the weights from the previous hidden layer to the current hidden layer,
 - **V**, the weights from the hidden layer to the output layer.



Training in simple RNNs: unrolling in time

Unlike feedforward networks:

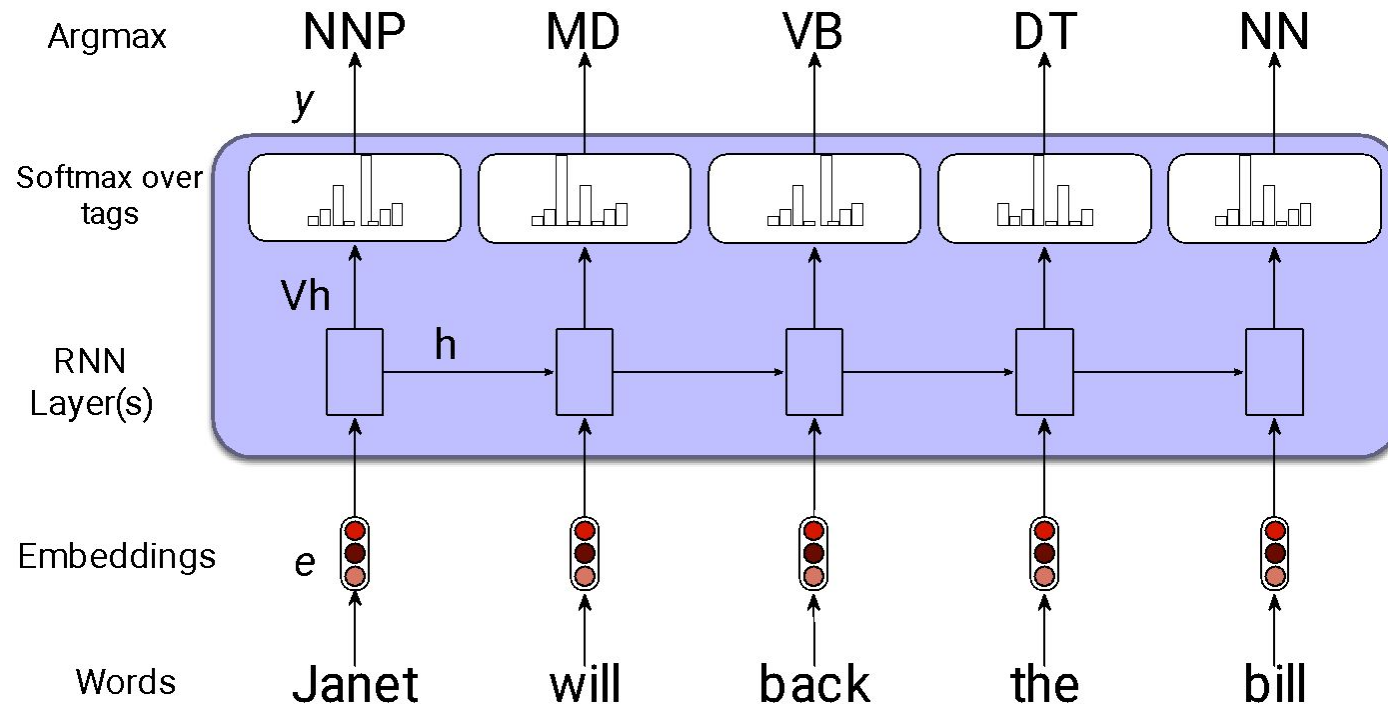
1. To compute loss function for the output at time t we need the hidden layer from time $t - 1$.
2. hidden layer at time t influences the output at time t and hidden layer at time $t+1$ (and hence the output and loss at $t+1$).



Examples of RNNs for Sequences

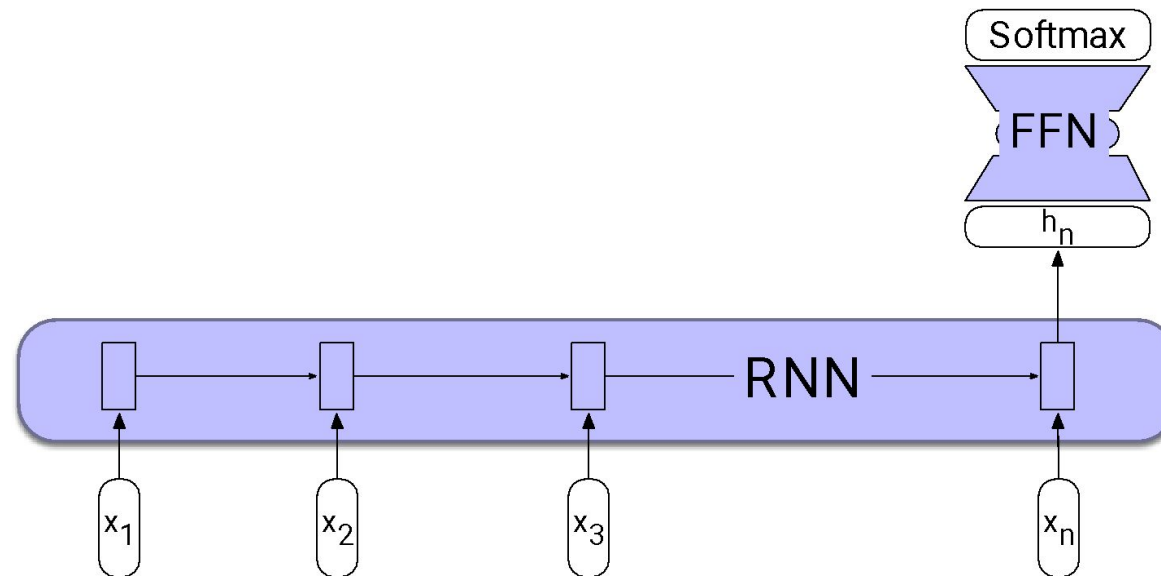
RNNs for sequence labeling

- Assign a label to each element of a sequence
- Part-of-speech tagging



RNNs for sequence classification

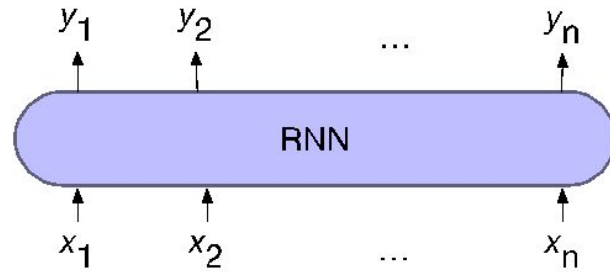
- Text classification



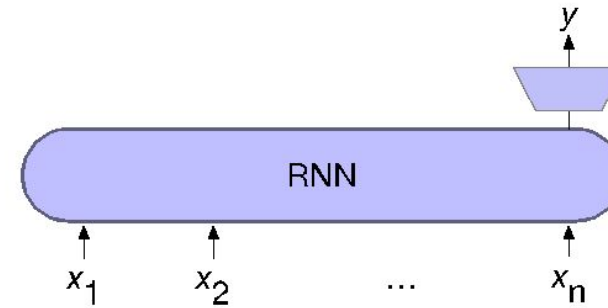
- Instead of taking the last state, could use some pooling function of all the output states, like **mean pooling**

$$h_{mean} = \frac{1}{n} \sum_{i=1}^n h_i$$

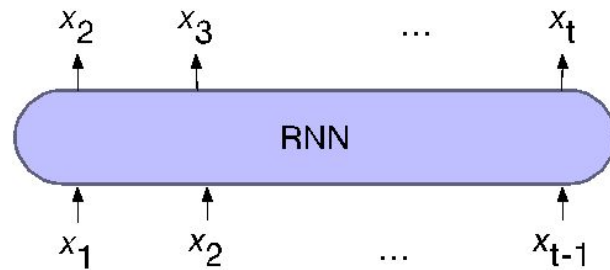
Four architectures for NLP tasks with RNNs



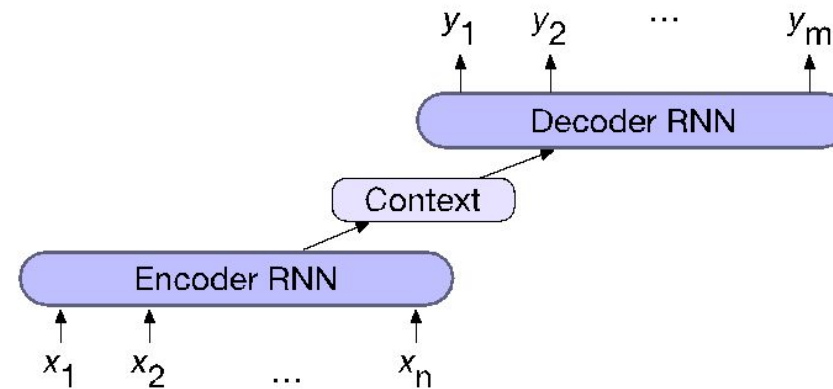
a) sequence labeling



b) sequence classification



c) language modeling



d) encoder-decoder

Advantages of RNN

- Can process sequential and temporal data effectively
- Maintains memory of previous inputs through hidden states
- Handles variable-length input and output sequences
- Useful for applications like language modeling, speech recognition, and time-series forecasting

Disadvantages of RNN

- Sequential processing makes training slower
- Struggles to learn long-term dependencies
- Suffers from vanishing and exploding gradient problems
- Difficult to parallelize
- Performance degrades on very long sequences

Questions?